Guía de Actividades Práctico-Experimentales Nro. 004

1. Datos Generales

Nombre del estudiante(s)	Israel Rodríguez Douglas Carreño Jossibel Perez Jorge Luzuriaga
Asignatura	Sistemas distribuidos
Ciclo	6to
Unidad	1
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	004
Título de la Práctica	Mecanismos de Comunicación Interproceso (IPC) y Red.
Nombre del Docente	José O. Guamán Q.
Fecha	13 de Octubre de 2026 16 de Octubre de 2026
Horario	Martes 10h30 – 11h30 Viernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Implementar un sistema de asignación de pedidos que integre comunicación entre procesos (colas IPC) y comunicación en red (sockets TCP), y utilizar un agente de IA local (Ollama) provisto por el docente para la asignación de tareas, evaluando su comportamiento frente a la carga y la distancia.

- Implementar comunicación entre procesos usando colas (IPC).
- Desarrollar comunicación entre computadoras mediante sockets TCP.
- Integrar un agente de decisión basado en IA usando Ollama.
- Simular un sistema real de asignación de pedidos (tipo delivery).

- Evaluar el comportamiento del sistema en función de la carga y la distancia.
- Analizar ventajas de usar IA frente a métodos tradicionales (reglas fijas).

3. Materiales, Reactivos:

- Guia practica
- 2 o más computadoras (misma red WiFi/LAN)
- Router o red local

4. Equipos y Herramientas

Hardware
- Computador

Software
- Lenguaje de programación (Python, java spring boot, node)
- IDE de programación
- Ollama / Open WebUI (servidor de modelos de la carrera, modo API)

5. Procedimiento / Metodología

Fase 0: analizar y revisar los códigos

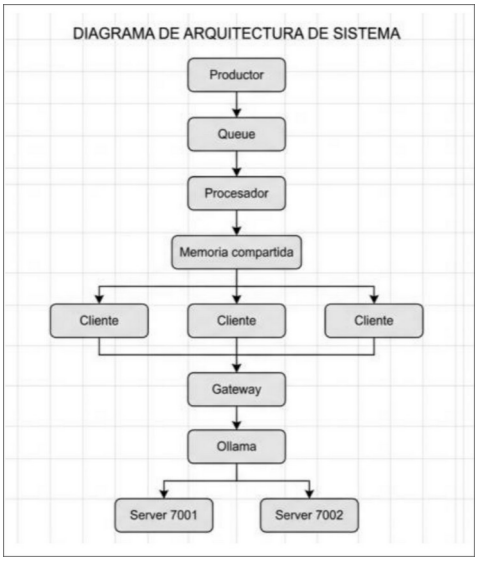


Fig 1: Arquitectura actual

Script actual	Ubicación	Función
app.py	cliente_system/	Es el punto de inicio actual del sistema. Inicia el sistema IPC y crea tres hilos clientes.
cliente.py	cliente_system/	Lee datos de la memoria compartida y los envía mediante TCP al Gateway.
main.py	cliente_system/ ipc_system/	Crea la cola IPC, la memoria compartida y los procesos productor/procesador.
productor.py	ipc_system/controls/	Genera datos simulados de temperatura y vibración y los coloca en la cola.
procesador.py	ipc_system/controls/	Extrae datos de la cola, calcula una condición de alerta y guarda el resultado en memoria compartida.
gateway.py	gateway-system/	Recibe datos de los clientes, consulta a Ollama y decide a qué servidor enviar los datos.
server.py	servidores-system/	Actúa como nodo servidor. Recibe los datos mediante TCP y los muestra en pantalla.

Se ejecutó el código base del repositorio en cuatro terminales (dos servidores, gateway y cliente) para comprender su funcionamiento antes de modificarlo.

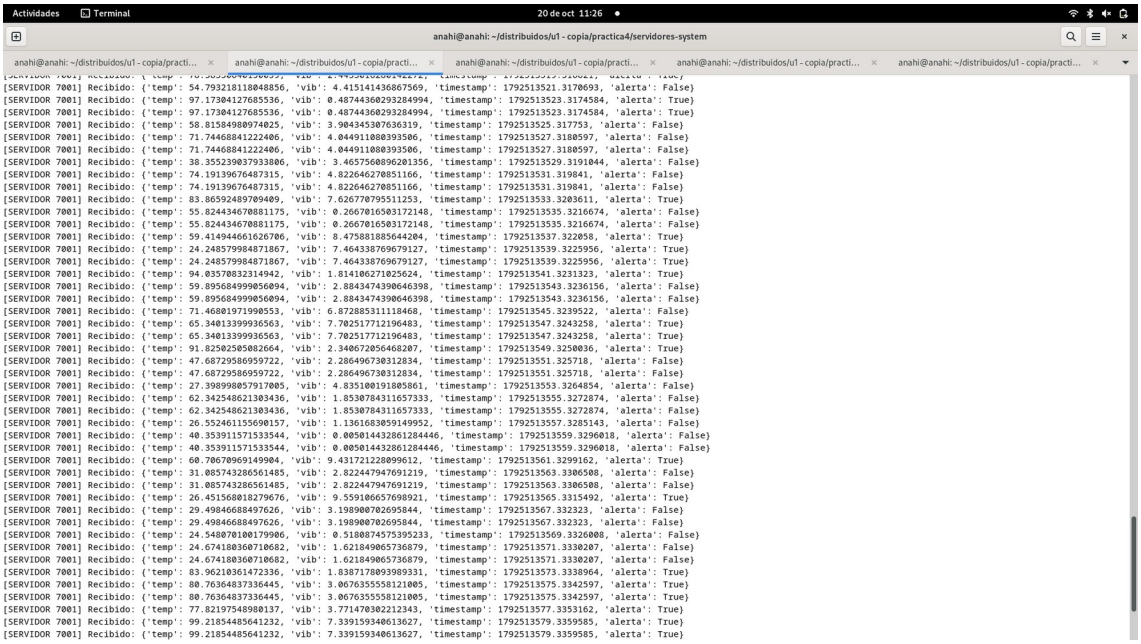


Fig 2: Servidor 7001 recibiendo datos de sensores (código base)

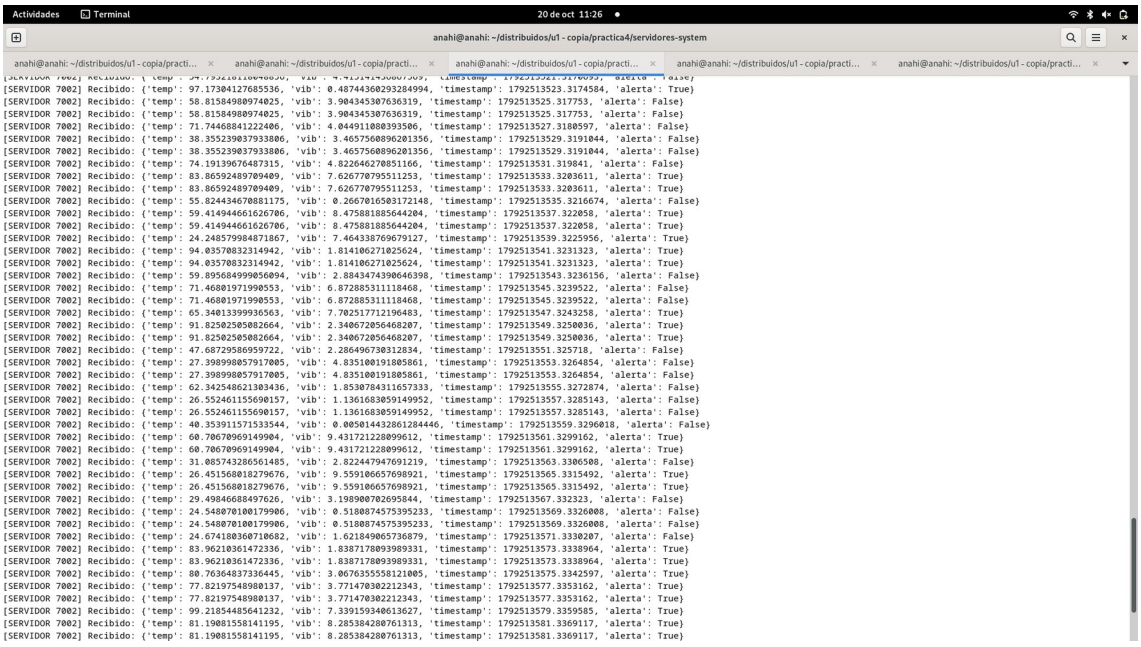


Fig 3: Servidor 7002 recibiendo datos de sensores (código base)

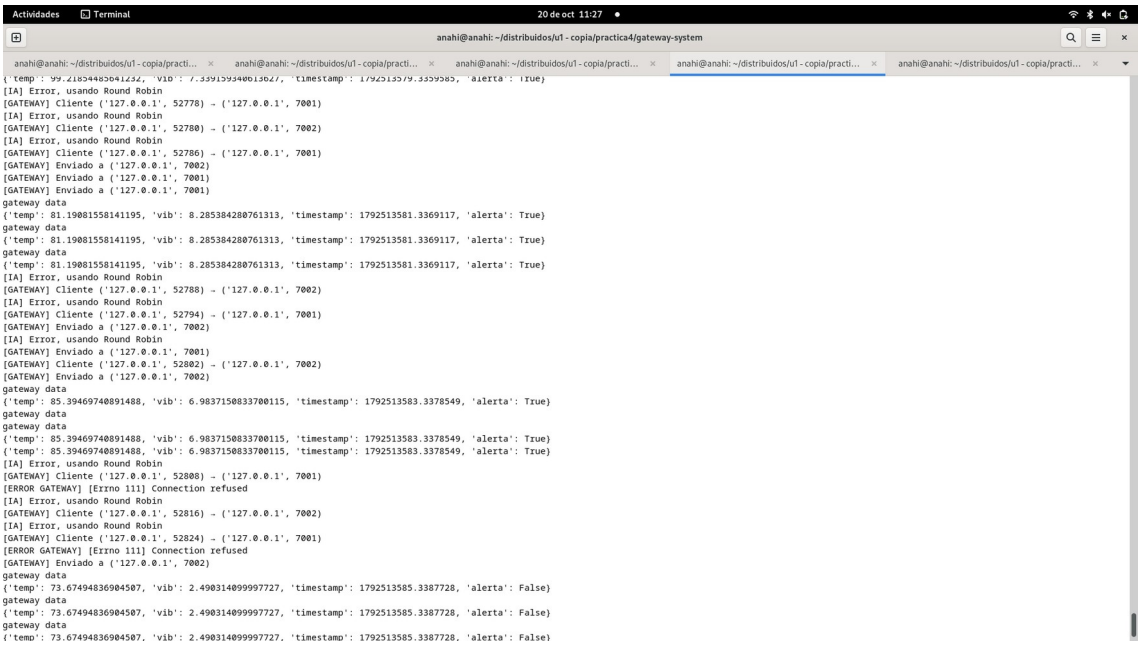


Fig 4: Gateway usando Round Robin y error "Connection refused" al detener el servidor 7002

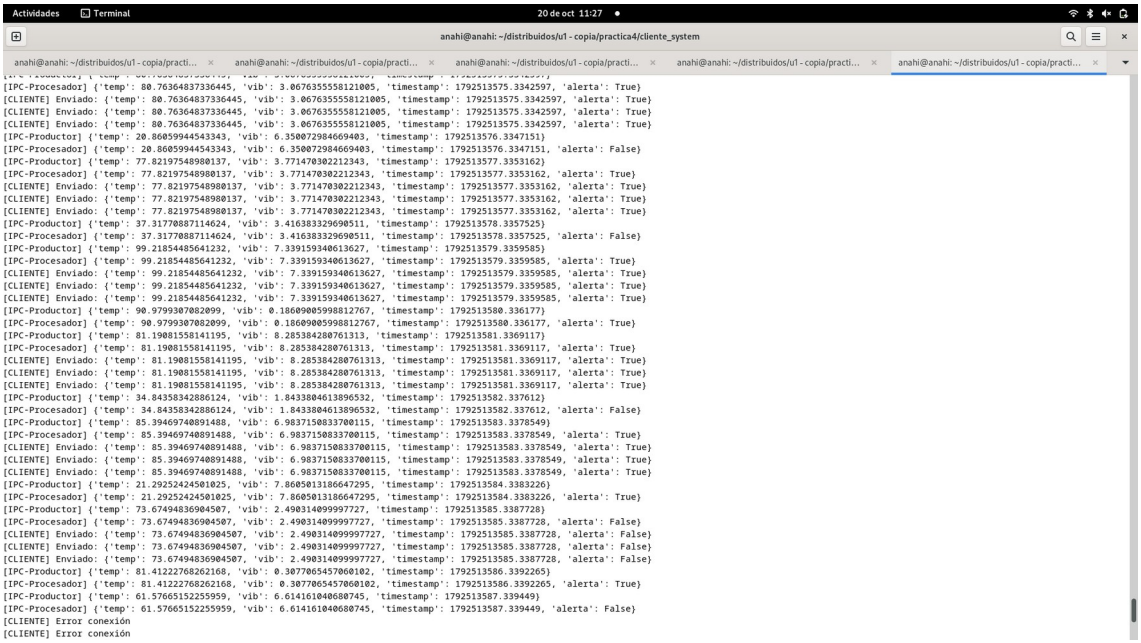


Fig 5: Cliente con los procesos productor y procesador (IPC)

Durante la ejecución se identificaron los siguientes problemas, que se corrigen en la Fase 2: (1) los tres hilos cliente leen el mismo dato de la memoria compartida, por lo que los servidores reciben el mismo timestamp repetido; (2) la memoria solo guarda el último dato, así que se pierden lecturas del productor; (3) la IA solo clasifica CRITICO/NORMAL y no conoce la carga ni la distancia de los servidores; (4) sin Ollama local el gateway usa Round Robin, y al detener el servidor 7002 aparece "Connection refused" y el dato se pierde sin reintento; (5) las direcciones IP están fijas en 127.0.0.1.

Fase 1: Preparación del entorno

Verificación de entorno: Python conexión con los servidores de la carrera modo API.

Se ingresó al servidor de modelos de la carrera (Open WebUI, http://10.20.202.199:3000), se generó una API key y se probó la conexión desde Python con el script prueba_api.py, que lista los modelos disponibles y realiza una consulta de prueba mediante los endpoints /api/models y /api/chat/completions con autenticación Bearer.

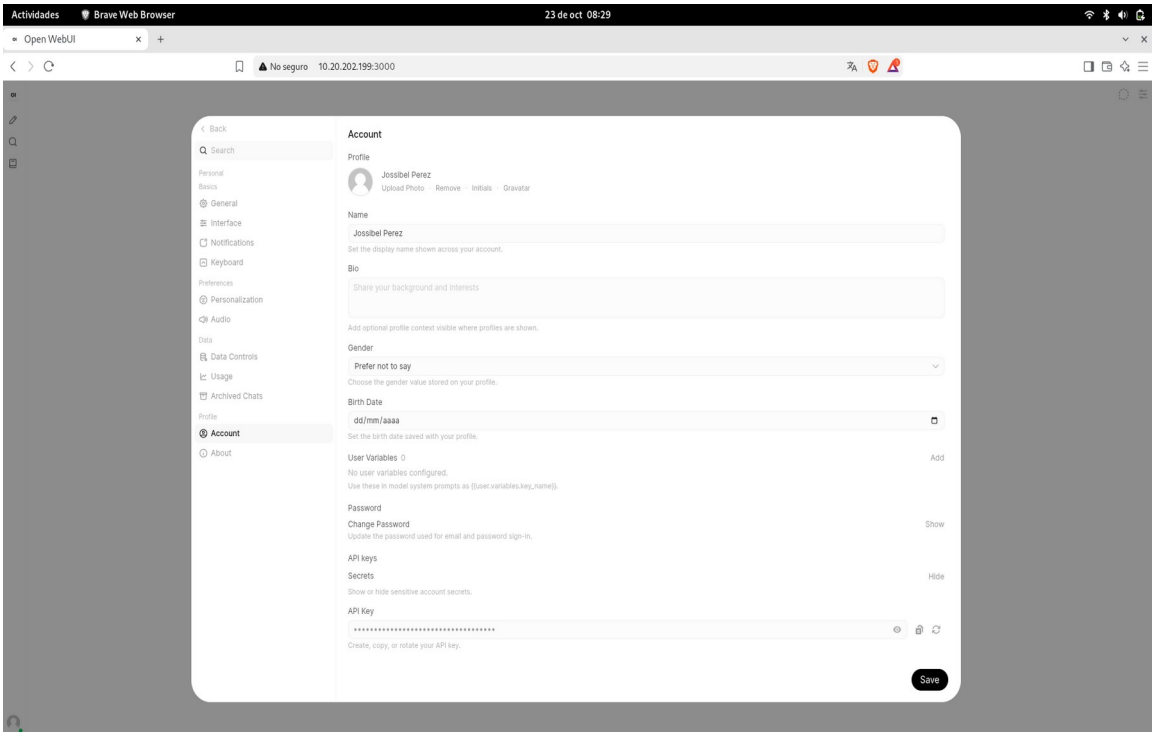


Fig 6: Generación de la API key en Open WebUI

```
anahi@anahi:~/distribuidos/u1 - copia/practica4$ python3 prueba_api.py
Estado: 200
Modelos disponibles:
  - qwen3-coder:30b
  - gemma3:12b
  - gemma3:4b
  - qwen3:8b

Probando con: qwen3-coder:30b
Respuesta: 7002
anahi@anahi:~/distribuidos/u1 - copia/practica4$
```

Fig 7: Conexión exitosa por API (Estado 200), modelos disponibles y respuesta del modelo

El servidor ofrece los modelos qwen3-coder:30b, gemma3:12b, gemma3:4b y qwen3:8b. Ante dos workers con cargas 3 y 1, el modelo eligió el worker 7002 (el de menor carga), lo que confirma que puede usarse como agente de decisión.

Fase 2: Implementación del sistema

Código actual	Nueva responsabilidad
productor.py	Generador de pedidos
procesador.py	Procesamiento de pedidos
gateway.py	Gateway de distribución
Parte IA de gateway.py	agente_ia.py
server.py	worker.py
cliente.py	Integrado en gateway.py (cliente TCP)
app.py	Convertido en main.py (punto de inicio único)
ipc_system/main.py	Integrado en main.py

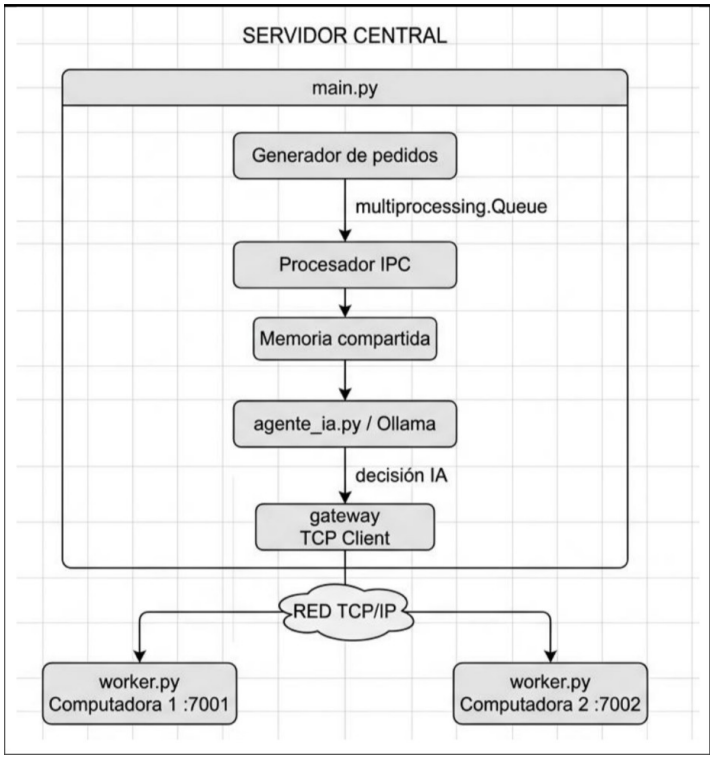


Fig 8: Arquitectura propuesta

1. IPC (cola de procesos).
   a. Crear procesos que generen pedidos.
   b. Usar multiprocessing.Queue para comunicación interna.

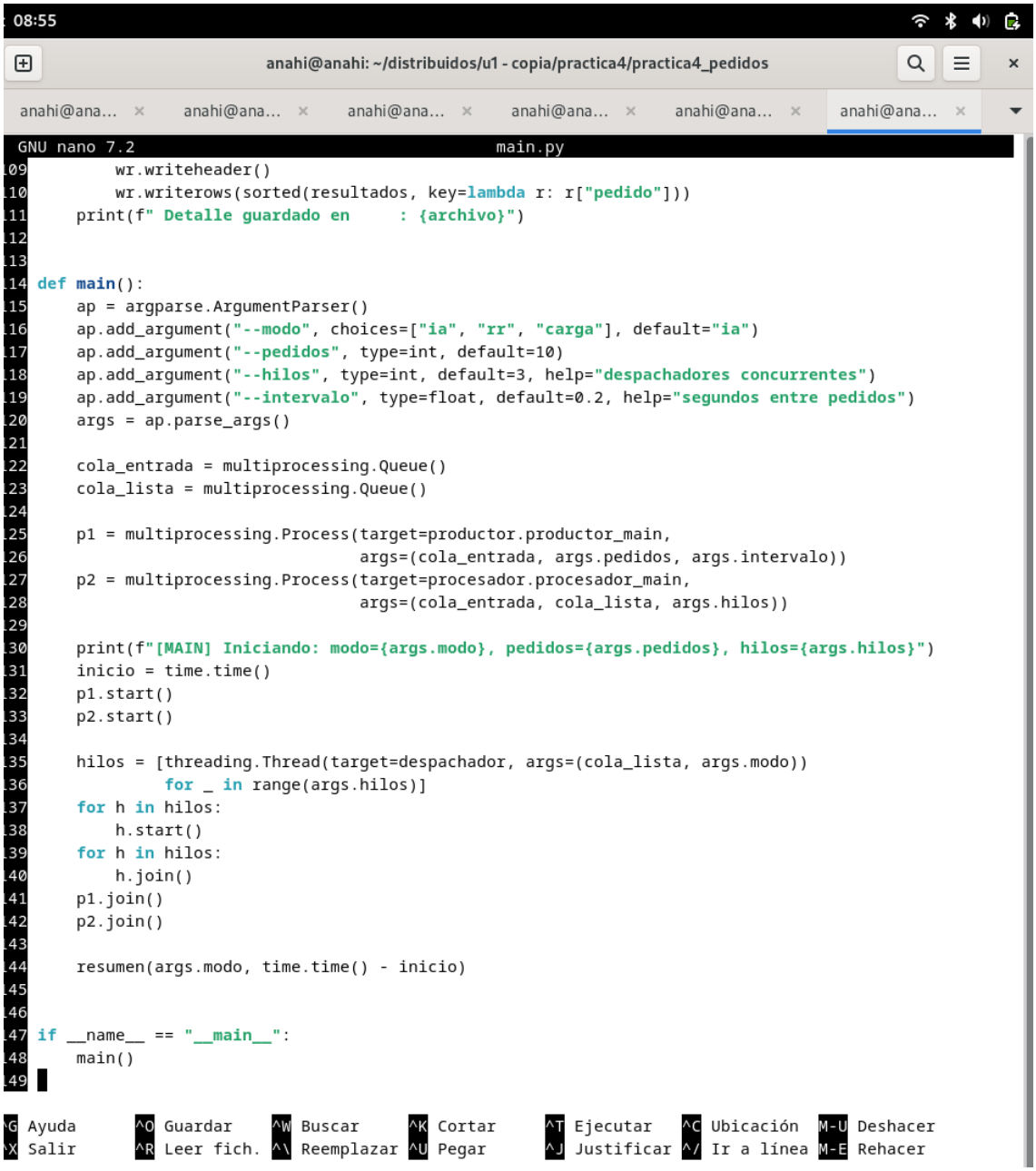


Fig 9: Colas IPC (multiprocessing.Queue) y procesos productor/procesador en main.py

El productor genera pedidos con identificador, zona, ubicación (x, y) y prioridad (ALTA o NORMAL); el procesador los valida y calcula su distancia en km a cada worker. Al usar colas en lugar de un único valor en memoria compartida se eliminan los datos duplicados y perdidos del código original.

2. Agente inteligente
   a. Implementar módulo agente_ia.py.
   b. Conectar con Ollama vía API.
   c. Diseñar prompt para toma de decisiones.

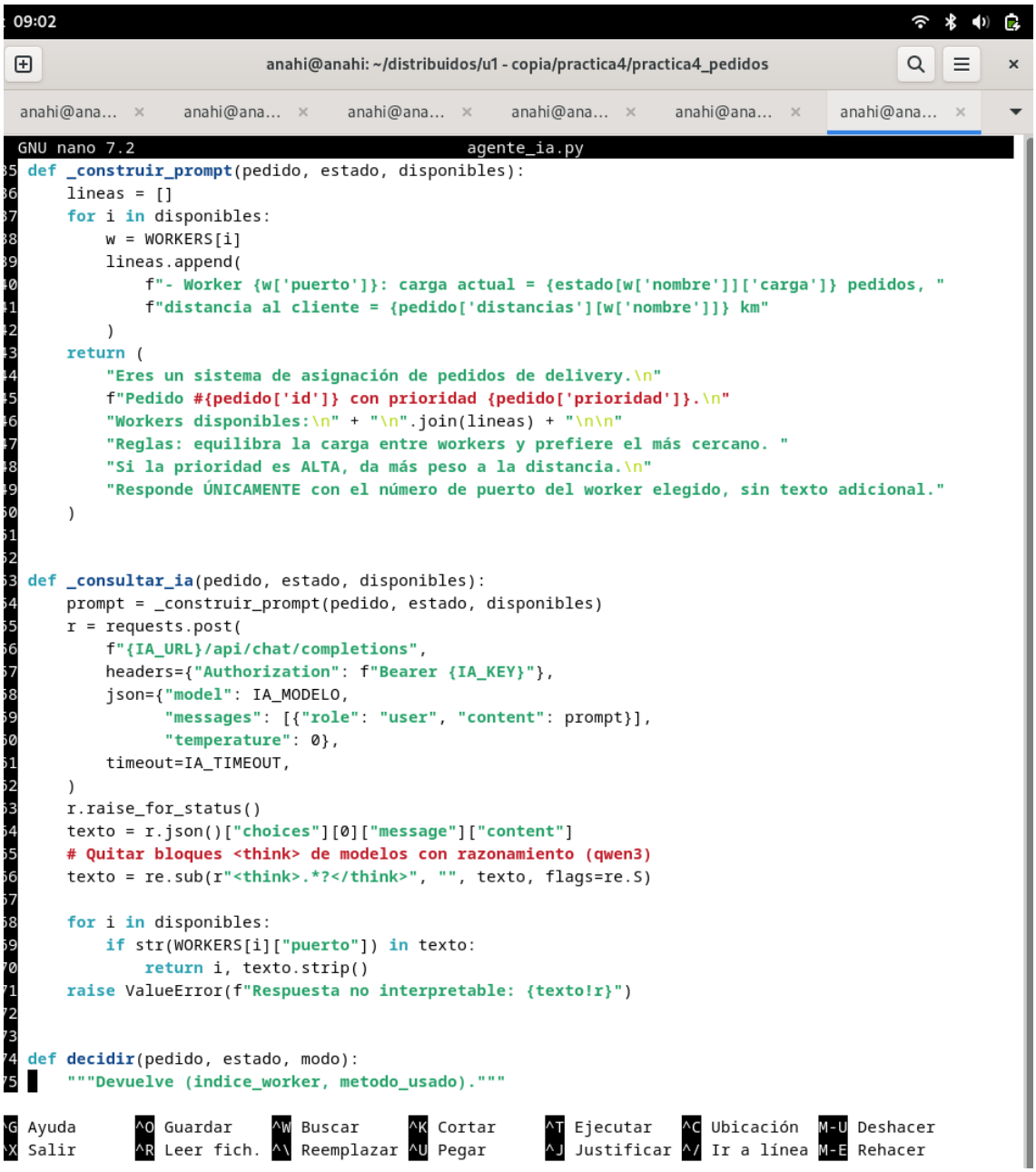


Fig 10: Diseño del prompt y conexión con el modelo vía API (agente_ia.py)

Para cada pedido, el agente envía al modelo (qwen3-coder:30b) la prioridad del pedido y, por cada worker disponible, su carga actual y su distancia al cliente; el modelo responde solo con el puerto del worker elegido. Si la consulta falla, se aplica la regla de menor carga como respaldo y ningún pedido queda sin asignar.

3. Comunicación en red
   a. Implementar sockets TCP.

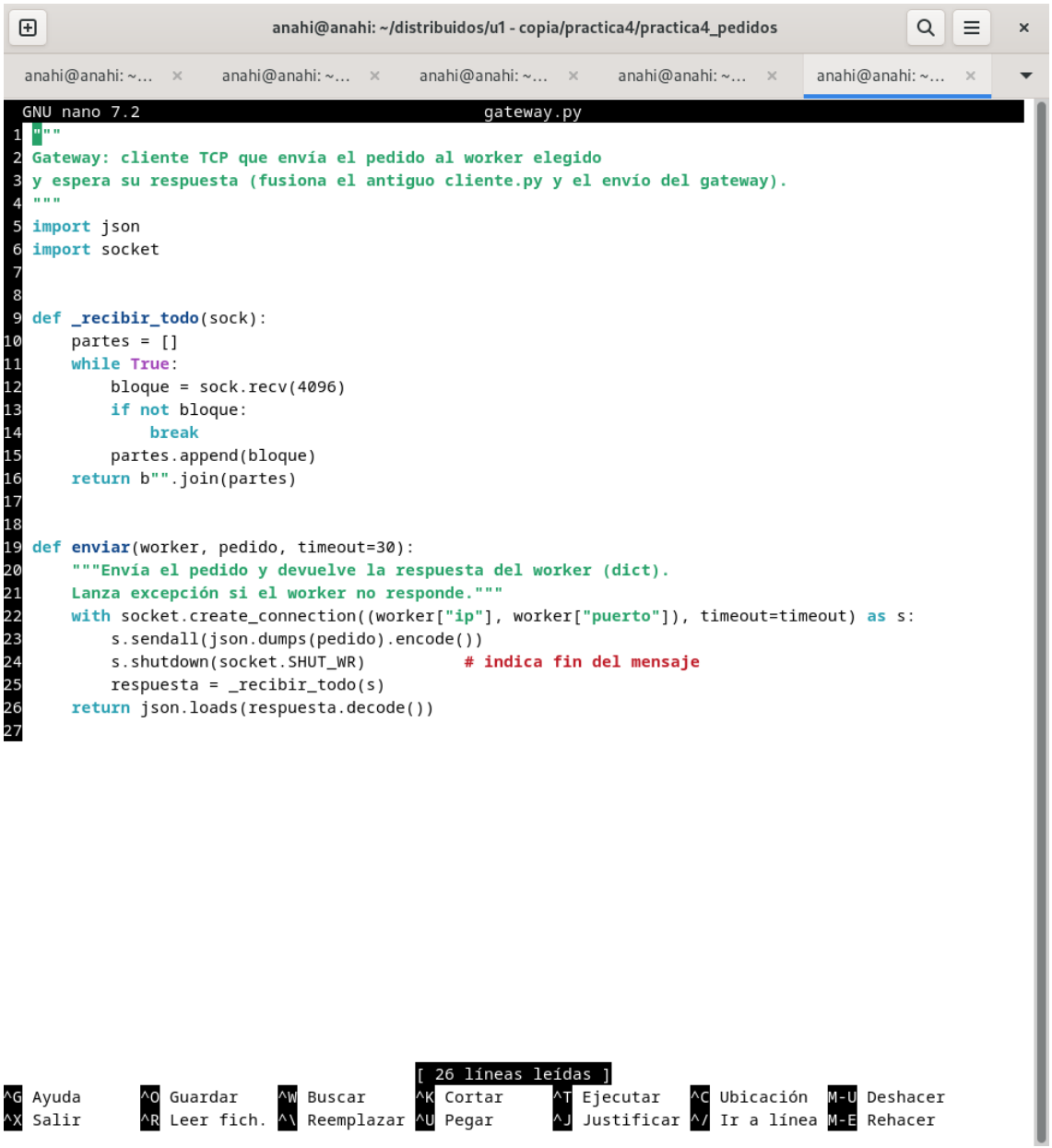


Fig 11: Cliente TCP del gateway: envía el pedido en JSON y espera la respuesta (gateway.py)

b. Configurar cliente (gateway) y servidor (worker).

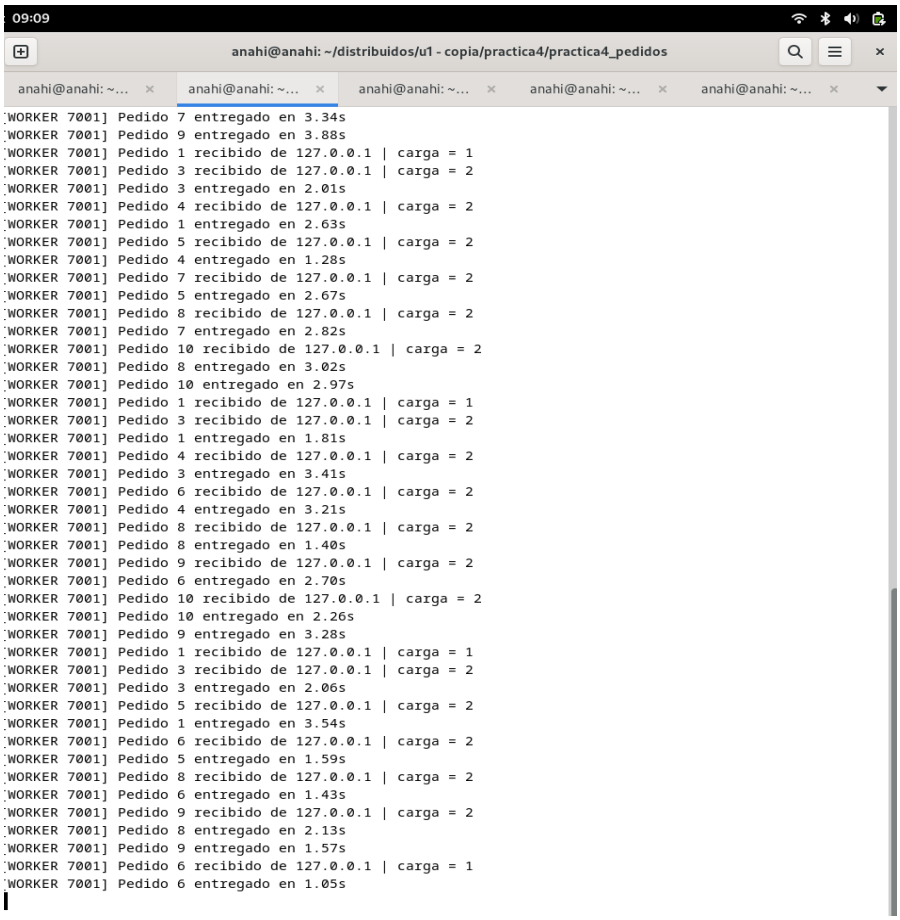
```
53 def main(puerto):
54     s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
55     s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
56     s.bind(("0.0.0.0", puerto))
57     s.listen()
58     print(f"[WORKER {puerto}] activo, esperando pedidos...")
59     while True:
60         conn, addr = s.accept()
61         threading.Thread(target=atender, args=(conn, addr, puerto), daemon=True).start()
62
63
64 if __name__ == "__main__":
65     main(int(sys.argv[1]))
66
```

```
^G Ayuda      ^O Guardar    ^W Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación  M-U Deshacer
^X Salir      ^R Leer fich. ^\ Reemplazar ^U Pegar      ^J Justificar ^/ Ir a línea M-E Rehacer
```

Fig 12: Servidor TCP del worker: bind, listen y accept con un hilo por pedido (worker.py)

Si un worker no responde, el gateway lo marca como inactivo y reasigna el pedido a otro worker disponible (tolerancia a fallos).

4. Nodo trabajador
   a. Ejecutar worker.py en cada computadora remota.
   b. Simular procesamiento de pedidos.



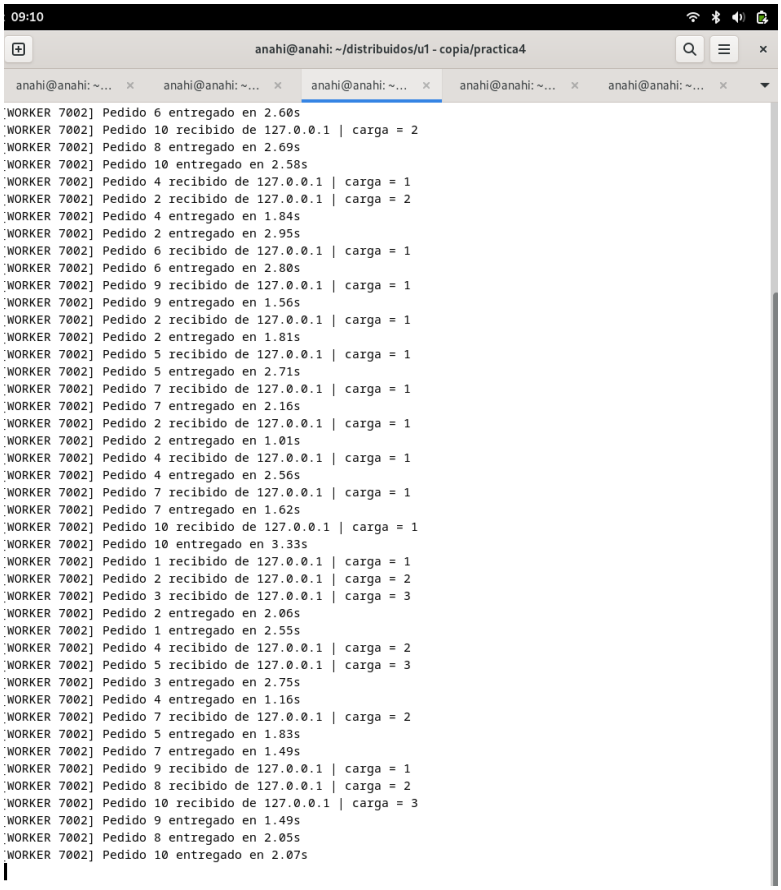


Fig 13: Workers 7001 y 7002 recibiendo y entregando pedidos

Cada worker incrementa su carga al recibir un pedido, simula la entrega con una demora proporcional a la distancia (0,5 s + 0,3 s por km) y responde con el estado ENTREGADO.

5. Integración total
   a. Ejecutar main.py en el servidor central.
   b. Verificar flujo completo:
   c. Generación → decisión IA → envío → ejecución

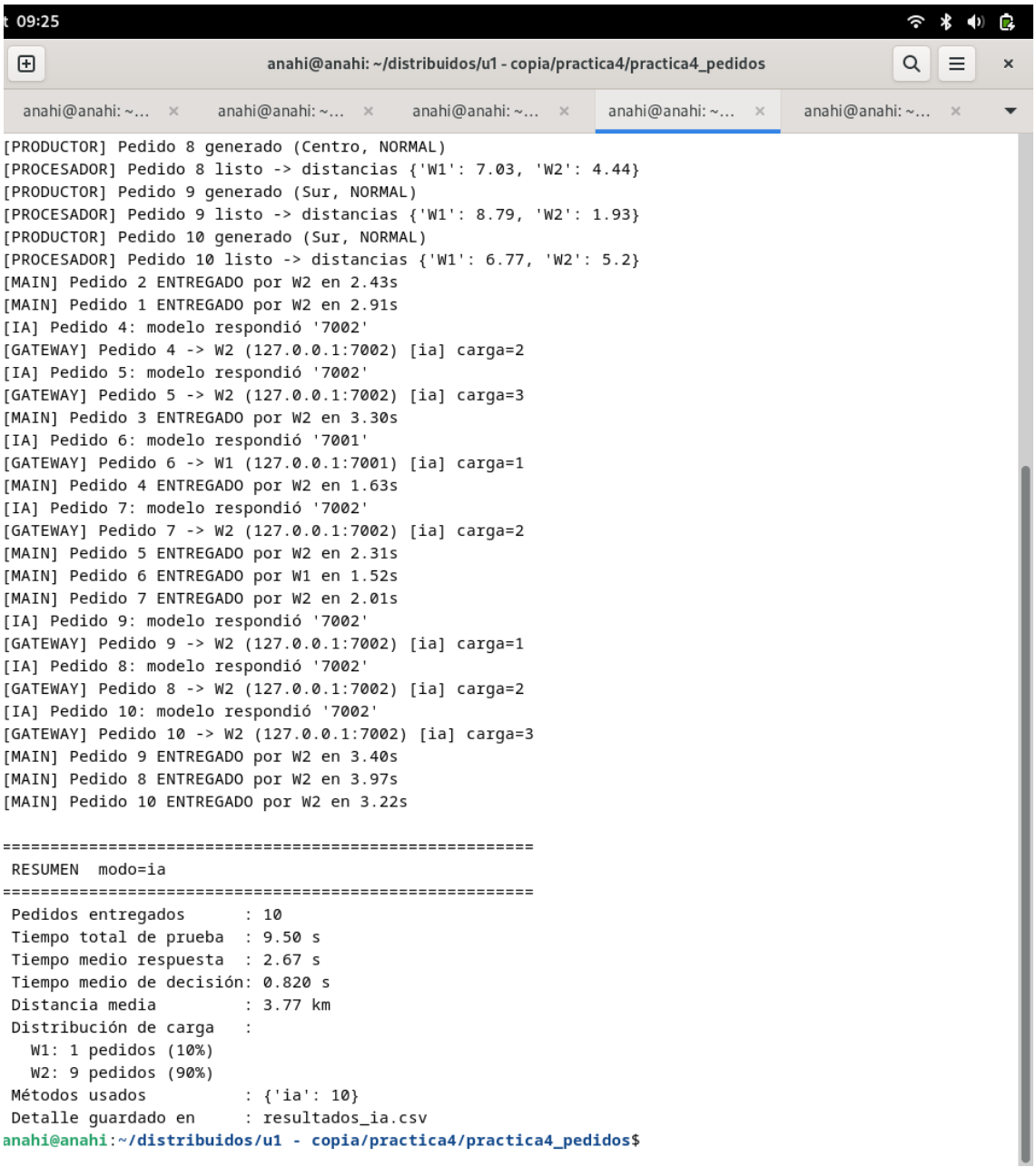


Fig 14: Ejecución completa en modo IA: generación → decisión IA → envío → entrega (10 pedidos)

Los 10 pedidos fueron decididos por el modelo (Métodos usados: ia = 10). El agente priorizó la cercanía: la distancia media fue de 3,77 km, aunque la carga quedó desbalanceada (W1: 10 %, W2: 90 %) porque la mayoría de los clientes estaban más cerca de W2. El tiempo medio de decisión fue de 0,82 s por pedido.

Fase 3: Pruebas

- Ejecutar múltiples pedidos simultáneos.
- Variar condiciones:
  - Distancia
  - Carga de nodos
  - Observar decisiones del agente IA.

Se ejecutaron pruebas con 30 pedidos generados cada 0,05 s y 6 hilos despachadores concurrentes, de modo que varios pedidos se procesan al mismo tiempo. Cada pedido tiene una ubicación aleatoria (distancia distinta a cada worker) y la carga de los workers cambia durante la prueba. La prueba se repitió con las dos reglas fijas; para el agente IA se toma como referencia la ejecución de la Fase 2 (Fig 14), realizada con 10 pedidos y 3 hilos.

```
python3 main.py --modo <rr | carga> --pedidos 30 --hilos 6 --intervalo 0.05
```

```
[MAIN] Pedido 15 ENTREGADO por W1 en 1.24s
[GATEWAY] Pedido 19 -> W1 (127.0.0.1:7001) [rr] carga=3
[MAIN] Pedido 14 ENTREGADO por W2 en 1.73s
[GATEWAY] Pedido 20 -> W2 (127.0.0.1:7002) [rr] carga=3
[MAIN] Pedido 13 ENTREGADO por W1 en 2.18s
[GATEWAY] Pedido 21 -> W1 (127.0.0.1:7001) [rr] carga=3
[MAIN] Pedido 16 ENTREGADO por W2 en 1.74s
[GATEWAY] Pedido 22 -> W2 (127.0.0.1:7002) [rr] carga=3
[MAIN] Pedido 18 ENTREGADO por W2 en 1.08s
[GATEWAY] Pedido 23 -> W1 (127.0.0.1:7001) [rr] carga=4
[MAIN] Pedido 23 ENTREGADO por W1 en 1.62s
[GATEWAY] Pedido 24 -> W2 (127.0.0.1:7002) [rr] carga=3
[MAIN] Pedido 17 ENTREGADO por W1 en 2.94s
[GATEWAY] Pedido 25 -> W1 (127.0.0.1:7001) [rr] carga=3
[MAIN] Pedido 22 ENTREGADO por W2 en 2.35s
[GATEWAY] Pedido 26 -> W2 (127.0.0.1:7002) [rr] carga=3
[MAIN] Pedido 19 ENTREGADO por W1 en 3.11s
[GATEWAY] Pedido 27 -> W1 (127.0.0.1:7001) [rr] carga=3
[MAIN] Pedido 20 ENTREGADO por W2 en 3.05s
[GATEWAY] Pedido 28 -> W2 (127.0.0.1:7002) [rr] carga=3
[MAIN] Pedido 21 ENTREGADO por W1 en 3.03s
[GATEWAY] Pedido 29 -> W1 (127.0.0.1:7001) [rr] carga=3
[MAIN] Pedido 24 ENTREGADO por W2 en 1.93s
[GATEWAY] Pedido 30 -> W2 (127.0.0.1:7002) [rr] carga=3
[MAIN] Pedido 28 ENTREGADO por W2 en 1.16s
[MAIN] Pedido 25 ENTREGADO por W1 en 2.97s
[MAIN] Pedido 26 ENTREGADO por W2 en 2.43s
[MAIN] Pedido 27 ENTREGADO por W1 en 2.50s
[MAIN] Pedido 29 ENTREGADO por W1 en 3.25s
[MAIN] Pedido 30 ENTREGADO por W2 en 2.76s

============================================================
 RESUMEN  modo=rr
============================================================
 Pedidos entregados      : 30
 Tiempo total de prueba  : 11.99 s
 Tiempo medio respuesta  : 2.13 s
 Tiempo medio de decisión: 0.000 s
 Distancia media         : 4.57 km
 Distribución de carga   :
   W1: 15 pedidos (50%)
   W2: 15 pedidos (50%)
 Métodos usados          : {'rr': 30}
 Detalle guardado en     : resultados_rr.csv
anahi@anahi:~/distribuidos/u1 - copia/practica4/practica4_pedidos$
```

Fig 15: Prueba con 30 pedidos simultáneos – Round Robin (regla fija)

```
[MAIN] Pedido 10 ENTREGADO por W1 en 3.16s
[GATEWAY] Pedido 19 -> W1 (127.0.0.1:7001) [carga] carga=3
[MAIN] Pedido 12 ENTREGADO por W2 en 3.07s
[GATEWAY] Pedido 20 -> W2 (127.0.0.1:7002) [carga] carga=3
[MAIN] Pedido 17 ENTREGADO por W2 en 1.03s
[GATEWAY] Pedido 21 -> W2 (127.0.0.1:7002) [carga] carga=3
[MAIN] Pedido 15 ENTREGADO por W1 en 1.64s
[GATEWAY] Pedido 22 -> W1 (127.0.0.1:7001) [carga] carga=3
[MAIN] Pedido 22 ENTREGADO por W1 en 1.81s
[GATEWAY] Pedido 23 -> W1 (127.0.0.1:7001) [carga] carga=3
[MAIN] Pedido 18 ENTREGADO por W1 en 2.53s
[GATEWAY] Pedido 24 -> W1 (127.0.0.1:7001) [carga] carga=3
[MAIN] Pedido 19 ENTREGADO por W1 en 2.30s
[GATEWAY] Pedido 25 -> W1 (127.0.0.1:7001) [carga] carga=3
[MAIN] Pedido 16 ENTREGADO por W2 en 3.73s
[GATEWAY] Pedido 26 -> W2 (127.0.0.1:7002) [carga] carga=3
[MAIN] Pedido 20 ENTREGADO por W2 en 2.95s
[GATEWAY] Pedido 27 -> W2 (127.0.0.1:7002) [carga] carga=3
[MAIN] Pedido 21 ENTREGADO por W2 en 2.90s
[GATEWAY] Pedido 28 -> W2 (127.0.0.1:7002) [carga] carga=3
[MAIN] Pedido 23 ENTREGADO por W1 en 1.34s
[GATEWAY] Pedido 29 -> W1 (127.0.0.1:7001) [carga] carga=3
[MAIN] Pedido 24 ENTREGADO por W1 en 1.46s
[GATEWAY] Pedido 30 -> W1 (127.0.0.1:7001) [carga] carga=3
[MAIN] Pedido 25 ENTREGADO por W1 en 2.43s
[MAIN] Pedido 27 ENTREGADO por W2 en 1.77s
[MAIN] Pedido 29 ENTREGADO por W1 en 1.41s
[MAIN] Pedido 26 ENTREGADO por W2 en 2.60s
[MAIN] Pedido 30 ENTREGADO por W1 en 2.71s
[MAIN] Pedido 28 ENTREGADO por W2 en 3.39s

============================================================
 RESUMEN  modo=carga
============================================================
 Pedidos entregados      : 30
 Tiempo total de prueba  : 12.59 s
 Tiempo medio respuesta  : 2.26 s
 Tiempo medio de decisión: 0.000 s
 Distancia media         : 4.90 km
 Distribución de carga   :
   W1: 16 pedidos (53%)
   W2: 14 pedidos (47%)
 Métodos usados          : {'carga': 30}
 Detalle guardado en     : resultados_carga.csv
anahi@anahi:~/distribuidos/u1 - copia/practica4/practica4_pedidos$
```

Fig 16: Prueba con 30 pedidos simultáneos – Menor carga (regla fija)

Fase 4: Análisis

- Comparar:
  - IA vs reglas fijas
- Evaluar:
  - Tiempo de respuesta
  - Distribución de carga

Con los resultados de la Fase 3 se comparó el agente IA frente a las reglas fijas, evaluando el tiempo de respuesta y la distribución de carga:

Métrica	IA (qwen3-coder:30b)*	Round Robin	Menor carga
Pedidos entregados	10	30	30
Tiempo total de prueba	9,50 s	11,99 s	12,59 s
Tiempo medio de respuesta	2,67 s	2,13 s	2,26 s
Tiempo medio de decisión	0,820 s	0,000 s	0,000 s
Distancia media	3,77 km	4,57 km	4,90 km
Distribución W1 / W2	1 / 9 (10 % / 90 %)	15 / 15 (50 % / 50 %)	16 / 14 (53 % / 47 %)

*Tabla 2: Comparación IA vs reglas fijas (reglas fijas: 30 pedidos y 6 hilos; *IA: 10 pedidos y 3 hilos, Fase 2)*

Round Robin: Round Robin alterna los workers, por lo que reparte la carga de forma exacta (50 % / 50 %) y decide de inmediato, pero ignora por completo la distancia (4,57 km de media).

Menor carga: Elige el worker con menos pedidos activos; la carga quedó casi equilibrada (53 % / 47 %) y también decide de inmediato, pero tampoco considera la distancia (4,90 km). Su tiempo medio de respuesta (2,26 s) fue similar al de Round Robin (2,13 s); la pequeña diferencia se debe a que cada prueba genera pedidos con ubicaciones aleatorias distintas.

Agente IA: En la prueba de la Fase 2 (10 pedidos), la IA redujo la distancia media a 3,77 km, pero concentró el 90 % de los pedidos en W2 y añadió 0,82 s por decisión. Como esa prueba tuvo menos pedidos y menos hilos, sus valores se comparan solo como referencia; aun así se ve que la IA fue la que más acortó la distancia, pero la que peor repartió la carga.

Comparación general: Las reglas fijas son instantáneas y equilibran la carga, pero no consideran la distancia; la IA puede combinar varios criterios (carga, distancia y prioridad), a cambio de un tiempo de decisión mayor y de depender de cómo esté redactado el prompt.

6. Resultados esperados:

- Correcta comunicación entre procesos (IPC).
- Envío de datos entre computadoras.
- Decisiones automáticas del agente IA.
- Distribución eficiente de pedidos.

El código base proporcionado por el docente está en el siguiente enlace: https://github.com/joseguaman1/distribuidos.git

7. Conclusiones:

- Se logró la comunicación entre procesos (IPC): el productor y el procesador intercambian pedidos mediante multiprocessing.Queue y, al reemplazar el único valor en memoria compartida por colas, se eliminaron los datos duplicados y perdidos del código original.
- La comunicación por sockets TCP entre el gateway y los workers funcionó correctamente: en todas las pruebas se entregaron el 100 % de los pedidos (30 de 30). Las pruebas se ejecutaron en una sola computadora (127.0.0.1); para usar computadoras distintas solo se cambia la IP de cada worker en config.py.
- Se integró el agente IA con el servidor de modelos de la carrera mediante su API (Open WebUI, qwen3-coder:30b). El modelo decidió cada asignación a partir de la carga y la distancia, y cuando la API no estuvo disponible (error 401) el sistema siguió entregando los pedidos con la regla de menor carga.
- Se cumplió el flujo completo del sistema de delivery: el pedido se genera, se procesa, el agente decide a qué worker enviarlo, se envía por TCP y el worker lo entrega.
- Las reglas fijas reparten la carga de forma equilibrada (50/50 con Round Robin y 53/47 con menor carga) y deciden de forma instantánea, pero no consideran la distancia (4,57 y 4,90 km de media).

- En la prueba de la Fase 2, la IA redujo la distancia media a 3,77 km, pero concentró el 90 % de los pedidos en un solo worker y tardó 0,82 s por decisión. La IA aporta valor cuando hay que combinar varios criterios, pero el prompt debe dar más peso a la carga y su tiempo de decisión es un costo que las reglas fijas no tienen.

8. Preguntas de Control

- **¿Qué diferencia hay entre la comunicación IPC (cola de procesos) y la comunicación por sockets TCP?**

La diferencia es que IPC se utiliza para la comunicación interna entre procesos que se ejecutan dentro de la misma computadora, mientras que los sockets TCP se utilizan para la comunicación entre distintas computadoras a través de una red.

Esto significa que la comunicación IPC es mucho más rápida que la comunicación por sockets TCP, ya que sucede en una misma computadora.

En esta práctica, el productor y el procesador se comunican con multiprocessing.Queue dentro del servidor central, mientras que el gateway y los workers se comunican con sockets TCP enviando cada pedido en formato JSON.

- **¿Por qué el agente IA y el generador de pedidos pueden ejecutarse en computadoras distintas sin compartir memoria?**

Porque no comparten memoria: se comunican enviando mensajes por la red. Cada pedido se serializa en JSON y se envía mediante un socket TCP o una petición HTTP a la dirección IP y puerto del otro equipo, que lo recibe y lo reconstruye en su propia memoria. En nuestra implementación, el agente consulta al modelo que está en otro servidor (10.20.202.199) por medio de su API, y el gateway envía los pedidos a los workers por TCP; ningún equipo necesita acceder a la RAM del otro.

- **¿Qué información debe viajar en el mensaje para que el agente decida correctamente? ¿Qué pasaría si faltara la carga del nodo?**

El mensaje debe incluir el identificador y la prioridad del pedido, la ubicación del cliente o su distancia a cada worker, la carga actual de cada worker (pedidos activos) y qué workers están disponibles. El prompt de agente_ia.py incluye exactamente estos datos.

Si falta la carga de nodo, el agente de IA no podría balancear el sistema. Mandaría pedidos a los nodos más cercanos sin saber si están saturados, lo que provocaría cuellos de botella y colapsaría el rendimiento de los servidores.

- **¿Qué ocurre si un worker se desconecta mientras hay pedidos en la cola?**

El gateway no logra conectarse con ese worker y recibe un error de conexión ("Connection refused"). En el código original ese dato se perdía, como se observó en la Fase 0. En nuestra implementación el gateway captura el error, marca al worker como inactivo y reasigna el pedido a otro worker disponible, de modo que los pedidos que siguen en la cola se reparten entre los workers que quedan; solo si no queda ninguno el pedido se registra como perdido.

- **¿En qué casos un agente de IA aporta valor frente a reglas fijas y en cuáles no?**

Aporta Valor: Cuando hay muchas variables enredadas a la vez (como equilibrar distancia, tráfico y batería al mismo tiempo). Ahí la IA piensa mejor.

No aporta valor: Cuando el problema es muy simple (como un "siempre manda al más cercano"). Para eso es mejor usar un código básico con if/else, que es instantáneo y no gasta recursos.

En nuestras pruebas, las reglas fijas decidieron de forma instantánea y con la carga equilibrada, mientras que la IA tardó en promedio 0,82 s por decisión y priorizó la distancia, lo que confirma que su ventaja depende del problema y del diseño del prompt.

9. Declaración de uso de IA

Para el desarrollo de la presente práctica se emplearon herramientas de Inteligencia Artificial con fines de apoyo académico y técnico. En particular, se utilizó NotebookLM como recurso para facilitar el entendimiento de la temática y la estructura conceptual de la práctica.

Prompt:

"Lee la Guía de Actividades Nro.4 de Sistemas Distribuidos y explica de forma simplificada y directa como responder las 5 preguntas de control sobre IPC, Sockets TCP y el agente Ollama."

Además, se utilizó Claude (Anthropic) como asistente para analizar el código base del docente, implementar y depurar el sistema de asignación de pedidos (main.py, productor.py, procesador.py, agente_ia.py, gateway.py y worker.py) y revisar la redacción del informe. El código fue ejecutado y verificado por el grupo, y las capturas corresponden a nuestras propias ejecuciones.

10. Bibliografía

[1] J. Guamán, "Guía de Actividades Práctico-Experimentales Nro. 004: Mecanismos de Comunicación Inter-Proceso (IPC) y Red," Sistemas Distribuidos, Semana 4, Universidad Nacional de Loja, Loja, Ecuador, 2026.

[2] A. S. Tanenbaum y M. van Steen, Distributed Systems, 4th ed. Maarten van Steen, 2023.

[3] Python Software Foundation, "multiprocessing — Process-based parallelism," Python 3 Documentation. [En línea]. Disponible: https://docs.python.org/3/library/multiprocessing.html

[4] Python Software Foundation, "socket — Low-level networking interface," Python 3 Documentation. [En línea]. Disponible: https://docs.python.org/3/library/socket.html

[5] Open WebUI, "API Endpoints," Open WebUI Documentation. [En línea]. Disponible: https://docs.openwebui.com/getting-started/api-endpoints

[6] Ollama, "Ollama API," GitHub. [En línea]. Disponible: https://github.com/ollama/ollama/blob/main/docs/api.md